Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-208БВ-25

Студент: Басаргин М.В.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 08.11.26

Москва, 2026

Постановка задачи

Цель работы: освоение принципов работы с файловыми системами и обеспечение обмена данными между процессами «File mapping» (разделяемая память и отображение её в адресное пространство процесса).

Задание: составить и отладить программу на языке C++, которая работает с процессами и организует взаимодействие между ними. Основной процесс создаёт для решения задачи дочерний процесс, а взаимодействие между процессами осуществляется через отображаемую память (shared memory и memory mapping), синхронизация доступа выполняется с помощью семафоров.

Вариант 3.

Пользователь вводит команды вида «число число число». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс производит деление первого числа на последующие, а результат выводит в файл. Если происходит деление на 0, то дочерний и родительский процесс завершают свою работу. Проверка деления на 0 должна осуществляться на стороне дочернего процесса. Числа имеют тип int. Количество чисел может быть произвольным.

Как работает parent

1. Выводит приглашение и читает с клавиатуры имя файла (системный вызов read из дескриптора 0).

2. Формирует уникальные имена, создаёт объект shared memory (shm_open с O_CREAT | O_EXCL), задаёт ему размер 4096 байт (ftruncate) и отображает в память (mmap). Затем создаёт два семафора cmd и resp со значением 0.

3. Делает fork. В дочернем процессе запускает программу child через execl, передавая ей имя файла и имена shared memory и семафоров. Путь к child родитель находит через readlink("/proc/self/exe"), поэтому child должен лежать в одной папке с parent.

4. В родительском процессе входит в цикл: читает строку с числами. Если строка не помещается в общую память, выводит сообщение и читает следующую. Иначе копирует строку в data, записывает её длину в len и вызывает sem_post(cmd).

5. Ждёт ответ ребёнка: вызывает sem_timedwait(resp) с таймаутом 100 мс. Если время вышло, проверяет через waitpid с WNOHANG, не умер ли ребёнок. Если умер, выводит сообщение об ошибке и выходит из цикла.

6. Читает status и решает, что делать: «k» – всё нормально, читаем следующую строку; «e» – ввод некорректный, выводит сообщение и продолжает; «z» – было деление на 0, выводит сообщение и завершает работу.

7. Пустая строка или Ctrl+D – выход из цикла. Если ребёнок ещё жив, родитель записывает в len значение UINT32_MAX, вызывает sem_post(cmd) и ждёт его через waitpid. В конце освобождает ресурсы: sem_close и sem_unlink для обоих семафоров, munmap, close и shm_unlink.

Как работает child

1. Получает в argv имя файла, имя shared memory и имена двух семафоров. Открывает файл через open (создаётся, если его нет, и очищается, если он есть), открывает объект shared memory (shm_open), отображает его (mmap) и открывает оба семафора (sem_open).

2. В цикле вызывает sem_wait(cmd) и спит, пока родитель не положит строку. Если в len лежит UINT32_MAX, выходит из цикла.

3. Копирует строку из общей памяти и разбирает её на числа типа int (функция parse). Если в строке есть что-то кроме чисел или число не помещается в int, записывает в status букву «e» и делает sem_post(resp).

4. Делит первое число на второе, результат на третье и так далее (целочисленное деление, считаю в long long, чтобы не было переполнения). Проверка на 0 происходит именно здесь, в child, перед каждым делением.

5. Если встретился 0, записывает в status букву «z», делает sem_post(resp) и завершается. Иначе записывает результат в файл (одно число на строку), записывает в status букву «k» и делает sem_post(resp).

6. Перед завершением закрывает семафоры (sem_close), снимает отображение памяти (munmap) и закрывает дескрипторы. Удаляет объекты (shm_unlink и sem_unlink) только родитель, потому что именно он их создал.

Все системные вызовы, которые могут завершиться ошибкой (shm_open, ftruncate, mmap, sem_open, fork, execl, open, write), проверяются. При ошибке программа выводит сообщение в стандартный поток ошибок, освобождает уже созданные объекты и завершается, чтобы в /dev/shm ничего не оставалось.

Код программы

parent.cpp

```
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <semaphore.h>
#include <errno.h>
#include <time.h>
#include <string>

#include <stdint.h>

#define SHM_SIZE 4096
#define MSG_EXIT UINT32_MAX

struct SharedData {
    uint32_t len;
    char     status;
    char     data[SHM_SIZE - sizeof(uint32_t) - 1];
};

static_assert(sizeof(SharedData) <= SHM_SIZE, "SharedData не помещается в SHM_SIZE");

static void print(int fd, const std::string& s) {
    const char* p = s.c_str();
    size_t left = s.size();
    while (left > 0) {
        ssize_t n = write(fd, p, left);
        if (n < 0) {
            if (errno == EINTR) continue;
            return;
        }
        p += n;
        left -= n;
    }
}

static bool read_line(std::string& line) {
    line.clear();
    char c;
    while (true) {
        ssize_t n = read(STDIN_FILENO, &c, 1);
        if (n < 0) {
            if (errno == EINTR) continue;
            return false;
        }
        if (n == 0) return !line.empty();
        if (c == '\n') return true;
        line += c;
    }
}

static std::string self_dir() {
    char path[4096];
    ssize_t len = readlink("/proc/self/exe", path, sizeof(path) - 1);
    if (len == -1) return ".";
    path[len] = '\0';
    while (len > 0 && path[len] != '/') --len;
    path[len] = '\0';
    return std::string(path);
}

static int wait_reply(sem_t* sem, pid_t pid, bool& exited) {
    while (true) {
        timespec ts;
        clock_gettime(CLOCK_REALTIME, &ts);
```

```
        ts.tv_nsec += 100 * 1000 * 1000;
        if (ts.tv_nsec >= 1000000000L) {
            ts.tv_sec++;
            ts.tv_nsec -= 1000000000L;
        }
        if (sem_timedwait(sem, &ts) == 0) return 0;
        if (errno == EINTR) continue;
        if (errno == ETIMEDOUT) {
            int st;
            if (waitpid(pid, &st, WNOHANG) == pid) {
                exited = true;
                return -1;
            }
            continue;
        }
        return -2;
    }
}

int main() {
    print(STDOUT_FILENO, "Введите имя файла: ");
    std::string fileName;
    if (!read_line(fileName) || fileName.empty()) {
        print(STDERR_FILENO, "Ошибка: имя файла не задано\n");
        return 1;
    }

    std::string suffix  = std::to_string(getpid());
    std::string shmName = "/lab3_shm_" + suffix;
    std::string semCmd  = "/lab3_cmd_" + suffix;
    std::string semResp = "/lab3_rsp_" + suffix;
    std::string childPath = self_dir() + "/child";

    int shm = shm_open(shmName.c_str(), O_RDWR | O_CREAT | O_EXCL, 0600);
    if (shm == -1) {
        print(STDERR_FILENO, "Ошибка: не удалось создать shared memory\n");
        return 1;
    }
    if (ftruncate(shm, SHM_SIZE) == -1) {
        print(STDERR_FILENO, "Ошибка: ftruncate не удался\n");
        close(shm);
        shm_unlink(shmName.c_str());
        return 1;
    }
    SharedData* sh = (SharedData*)mmap(nullptr, SHM_SIZE, PROT_READ | PROT_WRITE,
                                       MAP_SHARED, shm, 0);
    if (sh == MAP_FAILED) {
        print(STDERR_FILENO, "Ошибка: mmap не удался\n");
        close(shm);
        shm_unlink(shmName.c_str());
        return 1;
    }
    sh->len = 0;
    sh->status = 0;

    sem_t* cmd  = sem_open(semCmd.c_str(),  O_CREAT | O_EXCL, 0600, 0);
    sem_t* resp = sem_open(semResp.c_str(), O_CREAT | O_EXCL, 0600, 0);

    auto cleanup = [&]() {
        if (cmd != SEM_FAILED)  { sem_close(cmd);  sem_unlink(semCmd.c_str()); }
        if (resp != SEM_FAILED) { sem_close(resp); sem_unlink(semResp.c_str()); }
        munmap(sh, SHM_SIZE);
        close(shm);
        shm_unlink(shmName.c_str());
    };

    if (cmd == SEM_FAILED || resp == SEM_FAILED) {
        print(STDERR_FILENO, "Ошибка: не удалось создать семафор\n");
        cleanup();
        return 1;
    }
```

```
    pid_t pid = fork();
    if (pid == -1) {
        print(STDERR_FILENO, "Ошибка: fork не удался\n");
        cleanup();
        return 1;
    }
    if (pid == 0) {
        execl(childPath.c_str(), "child",
              fileName.c_str(), shmName.c_str(), semCmd.c_str(), semResp.c_str(),
              (char*)nullptr);
        print(STDERR_FILENO, "Ошибка: не удалось запустить child\n");
        _exit(1);
    }

    bool exited = false;
    print(STDOUT_FILENO, "Вводите числа через пробел (пустая строка или Ctrl+D для выхода):\n");

    const size_t maxLen = sizeof(sh->data);
    std::string line;
    while (read_line(line)) {
        if (line.empty()) break;
        line += '\n';
        if (line.size() > maxLen) {
            print(STDOUT_FILENO, "Слишком длинная строка, максимум " +
                                 std::to_string(maxLen - 1) + " символов\n");
            continue;
        }

        for (size_t i = 0; i < line.size(); i++) sh->data[i] = line[i];
        sh->len = (uint32_t)line.size();
        sem_post(cmd);

        int r = wait_reply(resp, pid, exited);
        if (r != 0) {
            print(STDERR_FILENO, "Дочерний процесс неожиданно завершился\n");
            break;
        }
        char status = sh->status;
        if (status == 'z') {
            print(STDOUT_FILENO, "Деление на 0. Завершение работы.\n");
            break;
        }
        if (status == 'e') {
            print(STDOUT_FILENO, "Некорректный ввод, ожидается: число число ...\n");
        }
    }

    if (!exited) {
        sh->len = MSG_EXIT;
        sem_post(cmd);
        int st;
        waitpid(pid, &st, 0);
    }

    cleanup();
    return 0;
}
```

child.cpp

```
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <semaphore.h>
#include <errno.h>
#include <string>
#include <vector>

#include <stdint.h>

#define SHM_SIZE 4096
#define MSG_EXIT UINT32_MAX

struct SharedData {
    uint32_t len;
    char     status;
    char     data[SHM_SIZE - sizeof(uint32_t) - 1];
};

static_assert(sizeof(SharedData) <= SHM_SIZE, "SharedData не помещается в SHM_SIZE");

static bool write_all(int fd, const std::string& s) {
    const char* p = s.c_str();
    size_t left = s.size();
    while (left > 0) {
        ssize_t n = write(fd, p, left);
        if (n < 0) {
            if (errno == EINTR) continue;
            return false;
        }
        p += n;
        left -= n;
    }
    return true;
}

static bool parse(const std::string& s, std::vector<int>& out) {
    out.clear();
    size_t i = 0, n = s.size();
    while (true) {
        while (i < n && (s[i] == ' ' || s[i] == '\t' || s[i] == '\r')) i++;
        if (i >= n) break;

        bool neg = false;
        if (s[i] == '-' || s[i] == '+') {
            neg = (s[i] == '-');
            i++;
        }
        if (i >= n || s[i] < '0' || s[i] > '9') return false;

        long long v = 0;
        while (i < n && s[i] >= '0' && s[i] <= '9') {
            v = v * 10 + (s[i] - '0');
            if (v > 2147483648LL) return false;
            i++;
        }
        if (i < n && s[i] != ' ' && s[i] != '\t' && s[i] != '\r') return false;
        if (neg) v = -v;
        if (v > 2147483647LL || v < -2147483648LL) return false;
        out.push_back((int)v);
    }
    return !out.empty();
}

int main(int argc, char* argv[]) {
    if (argc < 5) {
        write_all(STDERR_FILENO, "child: ожидаются аргументы: файл shm sem_cmd sem_resp\n");
        return 1;
```

```
}

int fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (fd == -1) {
    write_all(STDERR_FILENO, "child: не удалось открыть файл\n");
    return 1;
}

int shm = shm_open(argv[2], O_RDWR, 0);
if (shm == -1) {
    write_all(STDERR_FILENO, "child: не удалось открыть shared memory\n");
    close(fd);
    return 1;
}
SharedData* sh = (SharedData*)mmap(nullptr, SHM_SIZE, PROT_READ | PROT_WRITE,
                                   MAP_SHARED, shm, 0);
if (sh == MAP_FAILED) {
    write_all(STDERR_FILENO, "child: mmap не удался\n");
    close(shm);
    close(fd);
    return 1;
}
sem_t* cmd  = sem_open(argv[3], O_RDWR);
sem_t* resp = sem_open(argv[4], O_RDWR);
if (cmd == SEM_FAILED || resp == SEM_FAILED) {
    write_all(STDERR_FILENO, "child: не удалось открыть семафор\n");
    munmap(sh, SHM_SIZE);
    close(shm);
    close(fd);
    return 1;
}

int code = 0;
std::vector<int> nums;

while (true) {
    if (sem_wait(cmd) == -1) {
        if (errno == EINTR) continue;
        write_all(STDERR_FILENO, "child: ошибка sem_wait\n");
        code = 1;
        break;
    }

    if (sh->len == MSG_EXIT) break;

    std::string line(sh->data, sh->len);
    if (!line.empty() && line.back() == '\n') line.pop_back();

    if (!parse(line, nums)) {
        sh->status = 'e';
        sem_post(resp);
        continue;
    }

    long long result = nums[0];
    bool zero = false;
    for (size_t i = 1; i < nums.size(); i++) {
        if (nums[i] == 0) { zero = true; break; }
        result /= nums[i];
    }

    if (zero) {
        sh->status = 'z';
        sem_post(resp);
        break;
    }

    if (!write_all(fd, std::to_string(result) + "\n")) {
        write_all(STDERR_FILENO, "child: ошибка записи в файл\n");
```

```
            code = 1;
            break;
        }
        sh->status = 'k';
        sem_post(resp);
    }

    sem_close(cmd);
    sem_close(resp);
    munmap(sh, SHM_SIZE);
    close(shm);
    close(fd);
    return code;
}
```

Протокол работы программы

Тест 1. Обычная работа

```
$ ./parent
Введите имя файла: out.txt
Вводите числа через пробел (пустая строка или Ctrl+D для выхода):
100 5 2
-81 3 3
7 10 3
1000 10 10 10

$ cat out.txt
10
-9
0
1
```

Проверка вручную: 100 / 5 / 2 = 10; -81 / 3 / 3 = -9; 7 / 10 = 0 (деление целочисленное); 1000 / 10 / 10 / 10 = 1.

Тест 2. Деление на 0

```
$ ./parent
Введите имя файла: out2.txt
Вводите числа через пробел (пустая строка или Ctrl+D для выхода):
50 5
10 0 5
Деление на 0. Завершение работы.

$ cat out2.txt
10
```

Строка «10 0 5» в файл ничего не записала, оба процесса завершились сами.

Тест 3. Некорректный ввод

```
$ ./parent
Введите имя файла: out3.txt
Вводите числа через пробел (пустая строка или Ctrl+D для выхода):
abc
Некорректный ввод, ожидается: число число ...
12 x
Некорректный ввод, ожидается: число число ...
20 2
-20 -2
99999999999 1
Некорректный ввод, ожидается: число число ...

$ cat out3.txt
10
10
```

Некорректные строки (буквы и число, не помещающееся в int) пропускаются, программа продолжает работать.

Тест 4. Пустое имя файла

```
$ ./parent
Введите имя файла:
Ошибка: имя файла не задано
```

Тест 5. Две пары программ одновременно и очистка ресурсов

```
$ ./parent
Введите имя файла: a.txt

$ ./parent
Введите имя файла: b.txt

$ ls /dev/shm | grep lab3
lab3_shm_220
```

```
lab3_shm_221
sem.lab3_cmd_220
sem.lab3_cmd_221
sem.lab3_rsp_220
sem.lab3_rsp_221

(в обоих терминалах вводим числа и завершаем программы по Ctrl+D)

$ ls /dev/shm | grep lab3

$ cat a.txt
2
$ cat b.txt
3
```

Пока обе пары работают, в /dev/shm лежат два разных набора объектов (в имена входит PID родителя), конфликта нет. После завершения программ объекты удалены, в /dev/shm ничего не осталось. PID в имена подставляются свои, у вас значения будут другими.

Вывод

В этой лабораторной работе переделана программа из первой лабораторноой работы: вместо двух каналов (pipe) родитель и ребёнок теперь обмениваются данными через именованную разделяемую память. Родитель принимает от пользователя строки с числами и кладёт их в общую память, а ребёнок делит числа, пишет результат в файл и сообщает родителю результат одной буквой. Чтобы процессы не читали и не писали одновременно и знали, когда данные готовы, были использованы два именованных семафора.